AI - Assisted Coding-Assignment -19.3

HTNO:2303A52228
Batch:37

Lab 19 – Code Translation: Converting Between Programming Languages.

Task Description-1
Python to C++ Conversion.
Expected Output:
An equivalent working C++ class translated from Python.
Prompt:
Convert the following Python class into equivalent C++ code.
Ensure proper use of class, constructor, data types, and access specifiers.
Also provide a main function to test the class and produce the same output.
CODE AND OUTPUT :

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks
    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)
s = Student("Abhinay", 20, 85)
s.display()
```

```
Name: Abhinay
Age: 20
Marks: 85
```

student.cpp > ...

```cpp
#include <iostream>
using namespace std;
class Student {
private:
    string name;
    int age;
    int marks;
public:
    Student(string n, int a, int m) {
        name = n;
        age = a;
        marks = m;
    }
    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Marks: " << marks << endl;
    }
};
int main() {
    Student s("Abhinay", 20, 85);
    s.display();
    return 0;
}
```

```
Name: Abhinay
Age: 20
Marks: 85
```

Explanation:
In this task, you are using AI to convert Python code into C++.First, you take a Python class (like Student with name and marks) and give it to AI with a prompt. The AI will generate equivalent C++ code using concepts like class, constructor, and proper data types.After that, you run both programs (Python and C++) and check whether they give the same output.

Task Description-2
Java to Python Function Conversion.
Expected Output:
A correct Python function equivalent to the original Java prime-checking method.
Prompt:
Convert the following Java method that checks if a number is prime into Python. Ensure the Python version uses proper syntax and is easy to understand.Also include test cases with multiple inputs.
CODE AND OUTPUT :

```java
class PrimeCheck {
    public static void main(String[] args) {
        int n = 5;
        if (isPrime(n))
            System.out.println(n + " is Prime");
        else
            System.out.println(n + " is Not Prime");
    }
    static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i < n; i++) {
            if (n % i == 0)
                return false;
        }
        return true;
    }
}
```

```python
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
numbers = [1, 2, 3, 4, 5, 10, 13]
for num in numbers:
    print(num, "is prime?" , is_prime(num))
```

```
1 is prime? False
2 is prime? True
3 is prime? True
4 is prime? False
5 is prime? True
10 is prime? False
13 is prime? True
```

Explanation:
The Java program checks whether a number is prime or not. In the main method, a number is assigned to a variable and passed to the isPrime function. The isPrime method first checks if the number is less than or equal to 1, because such numbers are not prime. Then it uses a loop starting from 2 up to one less than the number, and checks if the number is divisible by any of these values. If it finds a divisor, it immediately returns false, meaning the number is not prime. If the loop completes without finding any divisor, the function returns true, indicating the number is prime.

Task Description-3
Pseudocode to Python Implementation.
Expected Output:
A working Python program that correctly implements the pseudocode logic.
Prompt:
Convert the following pseudocode for sorting numbers into Python code.
Ensure the logic is correctly implemented and include sample input and output.
CODE AND OUTPUT:

```
START
INPUT list of numbers
FOR i from 0 to n-1
    FOR j from 0 to n-i-2
        IF arr[j] > arr[j+1]
            SWAP arr[j] and arr[j+1]
END
PRINT sorted list
STOP
```

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
numbers = [5, 2, 9, 1, 5, 6]
print("Original:", numbers)
sorted_numbers = bubble_sort(numbers)
print("Sorted:", sorted_numbers)
```

```
\python.exe c:/Users/ajayr/Docum
Original: [5, 2, 9, 1, 5, 6]
Sorted: [1, 2, 5, 5, 6, 9]
```

Explanation:
In this task, the pseudocode represents a step-by-step logic for sorting numbers using a simple algorithm like Bubble Sort. Instead of writing code directly, the logic is first written in an easy-to-understand format using plain instructions such as loops, conditions, and swapping elements. This pseudocode is then given to an AI tool, which converts it into Python code by following the same logic. The generated Python program uses loops to compare adjacent elements and swap them if they are in the wrong order, gradually arranging the list in sorted form.

Task Description-4
SQL to Pandas Query.
Expected Output:
Equivalent Pandas query that retrieves the correct subset of data.
Prompt:
Convert the following SQL query into equivalent Pandas DataFrame code in Python. Ensure correct filtering and selection of columns, and include a sample DataFrame for testing.
CODE AND OUTPUT :

19.sql > ...

Run on active connection | Select block

```sql
SELECT name, salary FROM employees WHERE salary > 50000;
```

```python
import pandas as pd
data = {
    "name": ["A", "B", "C", "D"],
    "salary": [40000, 60000, 55000, 30000]
}
df = pd.DataFrame(data)
result = df[df["salary"] > 50000][["name", "salary"]]
print(result)
```

```
\python.exe c:/Users/ajay
  name  salary
1    B   60000
2    C   55000
```

Explanation:
In this task, a SQL query is converted into an equivalent Pandas operation in Python. The SQL query selects specific columns (name and salary) from a table where the salary is greater than 50,000. In Pandas, the same operation is performed using a DataFrame. First, sample dataset is created as a DataFrame. Then, filtering is applied by selecting only rows where the salary column is greater than 50,000. After filtering, only the required columns are selected using double brackets.

Task Description-5
Real-Time Application: Algorithm Translation Across Languages.
Expected Output:
Equivalent, tested code implementations in two different programming languages.
Prompt:
Convert the following Python searching algorithm into C language.Ensure the logic remains the same and provide a complete program with input and output.
CODE AND OUTPUT :

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
arr = [10, 20, 30, 40, 50]
key = 30
result = linear_search(arr, key)
if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

```
\python.exe c:/Users/ajayr/Do
Element found at index 2
```

> C 19.c > ...

```c
#include <stdio.h>
int linear_search(int arr[], int n, int key) {
    int i;
    for(i = 0; i < n; i++) {
        if(arr[i] == key) {
            return i;
        }
    }
    return -1;
}
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;
    int key = 30;
    int result = linear_search(arr, n, key);
    if(result != -1)
        printf("Element found at index %d", result);
    else
        printf("Element not found");
    return 0;
}
```

```
Element found at index 2
```

Explanation:
In this task, a searching algorithm written in Python is translated into C using AI. The original Python program uses a simple linear search to check each element in the list until the target value is found. The same logic is implemented in C, but with differences in syntax such as using arrays instead of lists, defining data types explicitly, and including a main function to run the program. During translation, challenges like manual memory handling, stricter syntax rules, and the need for header files in C are oserved.